深度学习
第 7 章　用 PyTorch 进行深度学习

——从激活函数到优化器，把神经网络的每个零件搞清楚

2026 年 3 月 17 日

本章题目导览：你将挑战哪些核心问题？

搭建神经网络就像组装一台精密机器：需要**激活函数**为输出引入非线性，需要**初始化策略**让训练一开始就走对方向，需要**损失函数**衡量预测有多准，还需要**优化器**一步步调整参数。本章 6 道题从零打通 PyTorch 深度学习的核心脉络：

- **Q7-01（辨析填空）**：四种激活函数横向对比——Sigmoid、Tanh、ReLU、Softmax，各自的公式、优缺点与适用场景是什么？
- **Q7-02（概念理解）**：参数初始化的重要性——为什么不能把权重全初始化为 0？Xavier 和 Kaiming 初始化解决了什么问题？
- **Q7-03（计算题）**：损失函数手动推导——给定预测值和标签，分别计算交叉熵损失和 MSE 损失，把公式变成具体数字。
- **Q7-04（对比分析）**：五种优化器对比——Momentum、AdaGrad、RMSProp、Adam 各自的更新规则和核心思想是什么？
- **Q7-05（代码阅读）**：自定义模型代码追踪——`nn.Module` 子类的数据流形状逐层追踪，`forward` 方法怎么运作？
- **Q7-06（综合应用）**：房价预测模型全流程——特征工程、模型搭建、损失函数选择和训练过程的完整实现思路。

1　第 7 章　用 PyTorch 进行深度学习

在前几章，我们已经理解了神经网络的基本原理：数据流入，经过若干层变换，输出预测结果，和标签对比，反向传播更新权重。

但”知道原理”和”真正能搭”之间，还差很多零件：

- 每一层之后要不要加激活函数？加哪种？
- 权重一开始怎么设置？全设为 0 行不行？
- 预测值和真实值的”差距”用什么公式来量化？

- 梯度算完了，用什么策略更新参数最有效？

这些问题，就是本章要解决的。**每一个零件都不是装饰品——选错了，模型可能根本训练不起来。**

先建立一个总感觉：PyTorch 搭网络像什么？

把搭建神经网络比作开一家餐厅。

- **激活函数** = 厨师的调味料：没有调味料，所有菜都是白水煮——网络没有激活函数，所有层叠加起来还是一个线性变换，表达能力极差；
- **参数初始化** = 餐厅开业前的备货：备太少（权重趋于 0）厨师无从下手，备太多（权重过大）食材腐坏——初始化决定训练能不能顺利起跑；
- **损失函数** = 顾客的评分系统：没有评分，厨师不知道菜好不好；评分维度选错，厨师改进方向也就跑偏了；
- **优化器** = 根据评分改进菜谱的策略：是只看今天的差评（SGD），还是记住历史教训（Momentum），还是对每道菜用不同力度改进（Adam）？

Q7-01　四种激活函数横向对比

题目描述：

文档第 7.1 节介绍了神经网络中常用的四种激活函数：Sigmoid、Tanh、ReLU 和 Softmax。在做题之前，先来认识一下它们的本质区别。

问题：

(1) 填写下表，将四种激活函数的关键特征补全：

激活函数	数学公式	主要缺点	典型应用场景
Sigmoid	$\sigma(x) = \dfrac{1}{1+e^{-x}}$	存在_____ 问题，输出非零均值	_____
Tanh	$\tanh(x) = \dfrac{e^x - e^{-x}}{e^x + e^{-x}}$	仍存在_____ 问题	输出需要零均值的_____
ReLU	$\mathrm{ReLU}(x) =$ _____	负输入区域存在_____ 现象	深层网络的_____ 层
Softmax	$p_i = \dfrac{e^{z_i}}{\sum_j e^{z_j}}$	输出之和为 1，适合_____	多分类任务的_____ 层

(2) ”梯度消失”（Vanishing Gradient）是什么？为什么 Sigmoid 和 Tanh 在深层网络中容易引发这个问题？

(3) ReLU 的全称是什么？它相比 Sigmoid/Tanh 的最大优势是什么（从计算效率和梯度角度回答）？

来源溯源：[文档第 7 章 7.1 激活函数]

小白必读：为什么神经网络必须要激活函数？

- **没有激活函数会发生什么？**假设有三层线性变换：$y = W_3(W_2(W_1x))$。由于矩阵乘法满足结合律，这等价于 $y = (W_3W_2W_1)x$，本质上还是一个线性变换！无论网络多深，表达能力和单层一模一样，完全无法拟合非线性的现实数据。
- **激活函数的本质**：在每一层输出后施加一个非线性变换，打破线性叠加的束缚，让网络有能力学习任意复杂的函数。
- **不同位置用不同激活函数**：隐藏层通常用 ReLU（计算快，梯度稳定）；二分类输出层用 Sigmoid（输出 0 到 1 之间的概率）；多分类输出层用 Softmax（输出加起来等于 1 的概率分布）；回归输出层通常不加激活函数（输出任意实数）。

生活比喻：激活函数像什么？

想象你在做一道判断题测验：

- **Sigmoid**：把任何分数都压缩到 $(0,1)$ 之间——”你有多大把握这道题是对的？”输出越接近 1，越有把握。但当分数特别高或特别低时，Sigmoid 的输出变化极小，梯度几乎为 0，反向传播时信号就”消失”了，深层网络因此难以更新。
- **ReLU**：正分保留，零分或负分直接归零——”这个特征有用就留，没用就扔”。简单粗暴，但计算极快，而且正值区域梯度恒为 1，信号可以畅通地传到最底层。
- **Softmax**：把一组原始分数变成”投票比例”——”三个候选答案，分别有 82%、11%、7% 的概率是对的”。结果之和必须等于 1，天然适合多分类。

参考答案与详细解析

(1) 激活函数对比表（填写内容）：

激活函数	数学公式	主要缺点	典型应用场景
Sigmoid	$\sigma(x) = \dfrac{1}{1+e^{-x}}$	梯度消失，输出非零均值	二分类输出层
Tanh	$\tanh(x) = \dfrac{e^x - e^{-x}}{e^x + e^{-x}}$	仍存在梯度消失问题	输出需要零均值的隐藏层
ReLU	$\max(0, x)$	负输入区域存在神经元死亡现象	深层网络的隐藏层
Softmax	$p_i = \dfrac{e^{z_i}}{\sum_j e^{z_j}}$	输出之和为 1，适合互斥分类	多分类任务的输出层

(2) 梯度消失原因：

Sigmoid 函数的导数为 $\sigma'(x) = \sigma(x)(1-\sigma(x))$，最大值仅为 0.25（在 $x = 0$ 时取得）。Tanh 的导数最大值为 1（在 $x = 0$ 时），远离原点后也迅速趋近于 0。

在反向传播中，梯度是逐层相乘的。若每层梯度最大为 0.25，经过 10 层后，梯度缩小至 $0.25^{10} \approx 10^{-6}$，几乎为 0，底层网络参数无法有效更新。

(3) ReLU 的优势：

ReLU（Rectified Linear Unit，修正线性单元）的公式为 $\max(0, x)$，优势体现在两方面：

- **计算极快**：只需判断正负，无需计算指数，比 Sigmoid/Tanh 快数倍；
- **梯度稳定**：正值区域梯度恒为 1，不会随层数加深而衰减，有效缓解梯度消失。

Q7-02 参数初始化：为什么不能全设为 0？

题目描述：

文档第 7.2 节介绍了多种参数初始化方法。一个常见的误区是”把所有权重初始化为 0，反正训练会自动调整”。

问题：

(1) **对称性破坏问题**：如果把一个全连接层（`nn.Linear`）的所有权重初始化为相同的值（包括全 0），会出现什么问题？从反向传播的角度解释”对称性”为什么无法被打破。

(2) **Xavier 初始化**：适用于 Sigmoid/Tanh 激活函数之前的层，权重从以下范围均匀采样：

$$W \sim \mathcal{U}\left(-\frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}, \frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}}\right)$$

若一个线性层输入特征数 $n_{\text{in}} = 128$，输出特征数 $n_{\text{out}} = 64$，计算 Xavier 初始化的采样范围上界（保留两位小数）。

(3) **Kaiming 初始化**：专为 ReLU 激活函数设计，权重从均值为 0、标准差为 $\sqrt{2/n_{\text{in}}}$ 的正态分布中采样。解释为什么 ReLU 需要专门的初始化策略（提示：ReLU 会”杀死”一半神经元）。

(4) 在 PyTorch 中，`nn.Linear` 层默认使用哪种初始化方式？如何手动将某层权重改为 Kaiming 正态初始化？写出相关代码。

来源溯源：[文档第 7 章 7.2 参数初始化和正则化]

小白必读：初始化为什么这么重要？

- **权重不是随便设的**：如果初始化过大，激活值和梯度会爆炸（梯度爆炸）；如果初始化过小，激活值趋近于 0，梯度也接近 0（梯度消失）。两种情况都会让训练无法有效进行。
- **全 0 初始化的致命问题**：如果同一层所有神经元的权重完全相同，那么正向传播时它们的输出完全相同，反向传播时它们接收到的梯度也完全相同，更新后仍然完全相同——这层神经元永远无法学到不同的特征，相当于只有一个神经元在工作。
- **为什么需要随机初始化**：随机初始化打破了对称性，让每个神经元从一开始就走向不同的”方向”，学到不同的特征。

参考答案与详细解析

(1) 对称性问题：

若所有权重 $w_{ij}^{(l)} = c$（常数），则同一层的每个神经元接收到完全相同的输入，产生完全相同的输出（正向传播对称）。反向传播时，损失对该层每个神经元的梯度也完全相同，参数更新幅度一致，导致更新后权重仍然相同——**对称性永远无法被打破**，整个层退化为”一个神经元”，无论宽度多大都形同虚设。

(2) Xavier 初始化上界计算：

$$\text{上界} = \frac{\sqrt{6}}{\sqrt{n_{\text{in}} + n_{\text{out}}}} = \frac{\sqrt{6}}{\sqrt{128+64}} = \frac{\sqrt{6}}{\sqrt{192}} = \frac{2.449}{13.856} \approx \boxed{0.18}$$

采样范围为 $[-0.18,\ 0.18]$。

(3) ReLU 需要专门初始化的原因：

ReLU 对所有负输入输出 0，相当于每次前向传播中约一半的神经元被”关闭”（输出为 0，梯度也为 0）。若使用 Xavier 初始化（针对输出方差设计），实际上只有约一半的神经元在传递信号，等效扇入减半，方差会缩小约 2 倍。Kaiming 初始化将标准差设为 $\sqrt{2/n_{\text{in}}}$（比 Xavier 多了因子 $\sqrt{2}$），恰好补偿了 ReLU 使方差减半的效果，使各层激活值的方差保持稳定。

(4) PyTorch 代码：

`nn.Linear` 默认使用 Kaiming 均匀初始化（`kaiming_uniform_`）。手动改为 Kaiming 正态初始化：

```python
import torch.nn as nn

layer = nn.Linear(128, 64)

# 手动将权重改为 Kaiming 正态初始化（适合 ReLU）
nn.init.kaiming_normal_(layer.weight, mode='fan_in',
                        nonlinearity='relu')
# 偏置通常初始化为 0
nn.init.zeros_(layer.bias)

```

常用初始化方法速查

- **全零初始化**：`nn.init.zeros_(tensor)` ——仅用于偏置，权重千万不要用
- **Xavier 均匀**：`nn.init.xavier_uniform_(tensor)` ——配合 Sigmoid/Tanh
- **Kaiming 正态**：`nn.init.kaiming_normal_(tensor, nonlinearity='relu')` ——配合 ReLU
- **正态分布**：`nn.init.normal_(tensor, mean=0, std=0.01)` ——通用小方差初始化

Q7-03 损失函数手动计算：交叉熵 vs MSE

题目描述：

文档第 7.4 节介绍了分类任务和回归任务的常用损失函数。损失函数是训练信号的来源——选错了，模型会朝错误的方向优化。

场景一：多分类任务（交叉熵损失）

一个三分类模型，对某样本输出的 logits 为 [2.0, 1.0, 0.1]，真实标签为第 0 类（即 $y = 0$）。

场景二：回归任务（MSE 损失）

一个房价预测模型，对 4 个样本的预测值（万元）为 $[\hat{y}_1, \hat{y}_2, \hat{y}_3, \hat{y}_4] = [102, 95, 310, 88]$，真实值为 $[100, 98, 300, 90]$。

问题：

(1) 对场景一，先计算 softmax 概率，再计算交叉熵损失：

$$\mathcal{L}_{\text{CE}} = -\log(p_y)$$

其中 p_y 是真实类别对应的预测概率。（结果保留两位小数）

(2) 对场景二，计算均方误差（MSE）损失：

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

（结果保留两位小数）

(3) PyTorch 的 `nn.CrossEntropyLoss` 内部已经包含了 Softmax 操作。如果在模型的输出层**已经手动加了 Softmax**，再使用 `nn.CrossEntropyLoss`，会产生什么问题？应该怎么修改？

(4) 为什么回归任务不能用交叉熵损失，分类任务不应该用 MSE 损失？各举一个具体理由。

来源溯源：[文档第 7 章 7.4 损失函数]

小白必读：损失函数是什么？为什么要分任务类型？

- **损失函数的本质**：量化”预测值和真实值有多不一样”的函数。训练就是通过梯度下降不断最小化损失函数的过程。
- **为什么要区分分类和回归？**分类任务的标签是离散类别（猫/狗/鸟），预测的是概率分布；回归任务的标签是连续数值（房价/温度），预测的是具体数值。两种情况下”误差”的计量方式完全不同。
- **CrossEntropyLoss 一定要配 logits 用**：PyTorch 的 `nn.CrossEntropyLoss` 内置了 softmax + log + 取均值的完整计算流程，输入必须是未归一化的原始分数（logits），而不是经过 softmax 后的概率。如果先 softmax 再送进去，相当于做了两次 softmax，结果会严重错误。

参考答案与详细解析

(1) 交叉熵损失计算：

第一步，计算 softmax 概率（logits $= [2.0, 1.0, 0.1]$）：

$$e^{2.0} \approx 7.389, \quad e^{1.0} \approx 2.718, \quad e^{0.1} \approx 1.105$$

$$\text{总和} = 7.389 + 2.718 + 1.105 = 11.212$$

$$p_0 = \frac{7.389}{11.212} \approx 0.659, \quad p_1 \approx 0.242, \quad p_2 \approx 0.099$$

第二步，计算交叉熵损失（真实类别为第 0 类，$p_y = p_0 \approx 0.659$）：

$$\mathcal{L}_{\text{CE}} = -\log(0.659) \approx \boxed{0.42}$$

(2) MSE 损失计算：

$$\mathcal{L}_{\text{MSE}} = \frac{(102-100)^2 + (95-98)^2 + (310-300)^2 + (88-90)^2}{4} = \frac{4+9+100+4}{4} = \frac{117}{4} = \boxed{29.25}$$

(3) 双重 Softmax 的问题：

`nn.CrossEntropyLoss` 内部等价于：$\mathcal{L} = -\log(\text{softmax}(\text{logits})_y)$。如果模型输出层已手动加了 softmax，那么传入的 $p = \text{softmax}(\text{logits})$，各元素在 $(0, 1)$ 之间，`CrossEntropyLoss` 再对它做一次 softmax，会把概率值当 logits 处理，使得概率分布被极度压平，损失计算完全失真，训练无法正常进行。

修改方案：

- **方案一（推荐）**：去掉模型输出层的 softmax，保持输出为 logits，直接配合 `nn.CrossEntropyLoss`；
- **方案二**：保留 softmax，将损失函数改为 `nn.NLLLoss`，并在输出后加 `torch.log`（因为 NLLLoss 期望输入 log-probabilities）。

(4) 损失函数不可乱用的原因：

- **回归不能用交叉熵**：交叉熵需要概率分布作为输入，而回归输出是任意实数，无法直接套用概率公式。强行使用会导致数学上无意义（对负数取对数会出错）。
- **分类不应用 MSE**：分类标签（如 $y \in \{0, 1, 2\}$）是无序的类别编号，MSE 会把类别间距离的大小当作重要信息处理，而实际上类别 0 和类别 2 并不比类别 0 和类别 1 ” 差距更大”。此外，MSE 在分类问题中梯度较小，训练收敛极慢。

Q7-04 五种优化器的核心思想对比

题目描述：

文档第 7.5 节介绍了多种参数更新方法。优化器决定了”梯度算出来之后，怎么用它来更新参数”。选错了优化器，训练可能收敛极慢，或者在最优解附近震荡。

问题：

(1) 填写下表，将五种优化器的关键特征补全：

优化器	核心更新规则（用文字描述）	主要优缺点
SGD	沿梯度负方向直接更新，步长为_____	简单，但容易陷入_____，收敛慢
Momentum	在 SGD 基础上累积_____，赋予参数”惯性”	加速收敛，但_____参数较多
AdaGrad	对每个参数使用_____学习率，历史梯度平方和大的参数学习率变____	自适应学习率，但学习率单调_____，后期更新极慢
RMSProp	改进 AdaGrad，用_____平均代替全量累积	解决了学习率_____问题
Adam	结合 Momentum 和 RMSProp，同时维护_____和_____的指数移动平均	效果好，但_____参数有时需调整

(2) **学习率衰减**：训练过程中为什么要让学习率随时间减小？常见的衰减策略有哪些（至少列举两种）？

(3) **Adam 的偏差修正**：Adam 的动量估计在初始阶段会偏向于 0，因此引入了偏差修正：

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

解释为什么初始阶段 m_t 会偏向于 0（提示：β_1 通常为 0.9，$m_0 = 0$，$m_1 = (1-\beta_1)g_1$）。

(4) 在 PyTorch 中，如何为网络不同层设置不同的学习率（例如：特征提取层学习率为 10^{-4}，分类头学习率为 10^{-3}）？写出关键代码片段。

来源溯源：[文档第 7 章 7.5 参数更新方法]

小白必读：优化器是什么？为什么不直接用梯度？

- **最简单的更新规则**：$w \leftarrow w - \eta \cdot g$，其中 η 是学习率，g 是梯度。这就是最基础的 SGD（随机梯度下降）。
- **为什么需要更复杂的优化器？**原始 SGD 有几个痛点：(1) 在平坦区域梯度很小，更新极慢；(2) 在陡峭区域梯度很大，更新容易震荡；(3) 所有参数共用同一个学习率，不够灵活。各种改进版优化器，就是针对这些痛点的解决方案。
- **Adam 是目前最常用的选择**：结合了动量（记住历史方向）和自适应学习率（对不同参数用不同步长），在绝大多数任务上开箱即用效果不错，是入门深度学习的默认选择。

生活比喻：五种优化器像五种下山策略

你在一座山上，要下到最低点（最优解），眼睛只能看到脚下的坡度（梯度）：

- **SGD**：每步严格沿当前最陡方向走固定步长，路线可能锯齿形剧烈震荡；
- **Momentum**：像滚下山的石头，有惯性，能越过小坑和轻微的鞍部，但可能冲过头；
- **AdaGrad**：在平坦路段迈大步，在陡峭路段迈小步——但走着走着，步子会越来越小，最终走不动；
- **RMSProp**：和 AdaGrad 类似，但”遗忘”很久以前的坡度，始终聚焦于近期信息，步子不会越来越小；
- **Adam**：既有滚石的惯性，又能根据近期坡度自动调节步长，是综合效果最好的”登山向导”。

参考答案与详细解析

(1) 优化器对比表（填写内容）：

优化器	核心更新规则	主要优缺点
SGD	沿梯度负方向直接更新，步长为学习率 η	简单，但容易陷入局部极小值，收敛慢
Momentum	累积历史梯度，赋予参数”惯性”	加速收敛，但超参数较多
AdaGrad	历史梯度平方和大的参数学习率变小（自适应）	自适应，但学习率单调递减，后期更新极慢
RMSProp	用指数移动平均代替全量累积	解决了学习率一直衰减问题
Adam	同时维护一阶动量和二阶动量的指数移动平均	效果好，但默认超参数有时需调整

(2) 学习率衰减策略：

训练初期需要大学习率以快速下降；训练后期接近最优解时，大学习率会导致在最优解附近震荡而无法收敛，需要小学习率精细调整。常见策略：

- **步进衰减（StepLR）**：每隔固定 epoch 数乘以衰减因子，如每 10 个 epoch 乘以 0.1；

- **余弦退火（CosineAnnealingLR）**：学习率按余弦函数平滑衰减，训练结束时接近 0；
- **指数衰减（ExponentialLR）**：每个 epoch 乘以固定衰减系数 γ，如 $\eta_t = \eta_0 \cdot \gamma^t$。

(3) Adam 偏差修正的原因：

$m_0 = 0$，$m_1 = \beta_1 m_0 + (1-\beta_1)g_1 = 0.1 \cdot g_1$。仅为真实梯度的 10%，严重低估了真实动量大小。偏差修正 $\hat{m}_1 = m_1/(1-\beta_1^1) = m_1/0.1 = g_1$，将其还原为真实梯度值。随着训练步数 t 增大，$\beta_1^t \to 0$，修正项趋近于 1，影响逐渐消失。

(4) 不同层设置不同学习率：

```python
import torch.optim as optim

model = MyModel()

# 分组设置不同学习率
optimizer = optim.Adam([
    {'params': model.feature_extractor.parameters(),
     'lr': 1e-4},
    {'params': model.classifier.parameters(),
     'lr': 1e-3}
], lr=1e-3)  # 默认学习率（作为 fallback）

# 之后正常训练即可
optimizer.zero_grad()
loss.backward()
optimizer.step()

```

Q7-05 自定义模型代码阅读：数据流形状追踪

题目描述：

文档第 7.3 节介绍了如何用 `nn.Module` 搭建自定义神经网络。以下是一个用于图像二分类的模型代码：

```python
import torch
import torch.nn as nn

class BinaryClassifier(nn.Module):
    def __init__(self, input_dim):
        super(BinaryClassifier, self).__init__()
        self.fc1 = nn.Linear(input_dim, 128)
        self.bn1 = nn.BatchNorm1d(128)
        self.dropout = nn.Dropout(p=0.3)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 1)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.bn1(self.fc1(x)))  # 第1步
        x = self.dropout(x)                   # 第2步
        x = self.relu(self.fc2(x))            # 第3步
        x = self.sigmoid(self.fc3(x))         # 第4步
        return x

model = BinaryClassifier(input_dim=784)

```

输入 x 的形状为 `[batch_size=32, input_dim=784]`（32 张展平的 28×28 图像）。

问题：

(1) 逐步追踪各阶段张量的形状：

a. 模型输入 x：_____

b. 第 1 步之后（`relu(bn1(fc1(x)))`）：_____

c. 第 2 步之后（`dropout(x)`）：_____

d. 第 3 步之后（`relu(fc2(x))`）：_____

e. 第 4 步之后（模型输出）：_____

(2) `nn.BatchNorm1d(128)` 的作用是什么？为什么要把它放在线性层**之后**、激活函数**之前**？

(3) `nn.Dropout(p=0.3)` 在**训练阶段**和**推理阶段**的行为有什么不同？在 PyTorch 中如何切换这两种模式？

(4) 如果将这个模型改为用 `nn.Sequential` 的写法，请写出等价的代码（忽略 Dropout）。

来源溯源：[文档第 7 章 7.3 搭建神经网络 / 7.2.8 Dropout]

小白必读：nn.Module 的两个核心方法

- **`__init__`：定义零件**。在这里声明所有要用到的层（`nn.Linear`、`nn.ReLU` 等）。PyTorch 会自动追踪这些层的参数，让优化器能找到它们。
- **`forward`：定义数据怎么流动**。调用 `model(x)` 时，实际上执行的就是 `forward(x)`。层的定义顺序（`__init__`）和使用顺序（`forward`）不必相同，这是 `nn.Module` 比 `nn.Sequential` 灵活的地方——可以实现跳跃连接、多输入、多输出等复杂结构。
- **不要忘记 `super().__init__()`**：继承 `nn.Module` 时，第一行必须调用父类初始化，否则参数注册机制会失效，优化器找不到参数。

参考答案与详细解析

(1) 逐层形状追踪：

步骤	操作	输出形状	说明
输入	`x`	`[32, 784]`	32 张展平的图像
第 1 步	`relu(bn1(fc1(x)))`	`[32, 128]`	Linear(784→128)，BN，ReLU
第 2 步	`dropout(x)`	`[32, 128]`	Dropout 不改变形状
第 3 步	`relu(fc2(x))`	`[32, 64]`	Linear(128→64)，ReLU
第 4 步	`sigmoid(fc3(x))`	`[32, 1]`	Linear(64→1)，Sigmoid

(2) BatchNorm1d 的作用及位置：

批归一化（Batch Normalization）对每一批数据的每个特征维度独立进行归一化（减去均值，除以标准差），使激活值保持在稳定的范围内，有助于加速训练、缓解梯度问题，并起到一定的正则化作用。放在线性层**之后**、激活函数**之前**，是因为：归一化的对象是线性变换的输出（未激活的值），使其接近标准正态分布，再经过激活函数时梯度信号最稳定。若放在激活函数之后，ReLU 已经截断了负值，归一化效果会大打折扣。

(3) Dropout 在训练 vs 推理阶段的差异：

- **训练阶段**：以概率 $p = 0.3$ 随机将 30% 的神经元输出置为 0，迫使网络不依赖某几个特定神经元，从而起到正则化作用。未被置零的神经元输出会乘以 $\frac{1}{1-p}$ 进行缩放，保持期望值不变。
- **推理阶段**：所有神经元正常输出，不做任何置零操作（因为推理时需要稳定的确定性输出）。

在 PyTorch 中通过以下方式切换：

```
model.train()    # 切换为训练模式（Dropout 生效）
model.eval()     # 切换为推理模式（Dropout 关闭）

```

(4) 等价的 Sequential 写法：

```
model = nn.Sequential(
    nn.Linear(784, 128),
    nn.BatchNorm1d(128),
    nn.ReLU(),
    nn.Linear(128, 64),
```

```
    nn.ReLU(),
    nn.Linear(64, 1),
    nn.Sigmoid()
)

```

nn.Module vs nn.Sequential 的选择建议

- **用 Sequential**：结构简单，数据单路径从头流到尾，代码更简洁；
- **用 Module 子类**：需要跳跃连接（ResNet）、多分支（Inception）、多输入或多输出，或者 forward 中有条件逻辑时，必须用 Module 子类。

Q7-06 综合应用：房价预测全流程分析

题目描述：

文档第 7.6 节给出了一个用神经网络预测房价的完整案例。以下是简化后的核心代码骨架：

```python
import torch
import torch.nn as nn
import pandas as pd
from sklearn.preprocessing import StandardScaler

# ---- 7.6.2 特征工程 ----
df = pd.read_csv('house_prices.csv')
# 处理缺失值：数值列用均值填充，类别列用众数填充
num_cols = df.select_dtypes(include='number').columns
cat_cols = df.select_dtypes(exclude='number').columns
df[num_cols] = df[num_cols].fillna(df[num_cols].mean())
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])
# 类别列独热编码
df = pd.get_dummies(df, columns=cat_cols)
# 特征标准化
scaler = StandardScaler()
X = scaler.fit_transform(df.drop('SalePrice', axis=1))
y = df['SalePrice'].values

# ---- 7.6.3 搭建模型 ----
class HousePriceModel(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 1)          # 输出层无激活函数
        )

    def forward(self, x):
        return self.net(x)

# ---- 7.6.4 损失函数 ----
loss_fn = nn.MSELoss()

# ---- 7.6.5 模型训练 ----
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

```

问题：

(1) **特征工程**：代码中对类别特征进行了”独热编码”（One-Hot Encoding）。解释独热编码的原理，并

小白必读：训练一个神经网络，到底在做什么？

- **一次训练迭代的四步节奏**：(1) 前向传播——数据流过网络，得到预测值；(2) 计算损失——预测值和真实值的差距；(3) 反向传播——自动计算所有参数的梯度；(4) 参数更新——优化器按梯度方向调整参数。
- **为什么要多次迭代？**一次迭代通常只用一小批数据（mini-batch），神经网络需要看成千上万次数据才能学会规律。每轮完整过一遍数据集叫一个 epoch，通常训练需要几十到几百个 epoch。
- **训练集、验证集、测试集的区别**：训练集用来更新参数；验证集用来调超参数（学习率、网络结构），判断是否过拟合；测试集只在最终评估时用一次，不参与任何调整。

生活比喻：训练神经网络就像备考

- **训练集** = 你平时刷的题目，做完对答案，每次都学到东西（更新权重）；
- **验证集** = 模拟考，用来判断自己当前水平，决定要不要换一本参考书（调超参数）；
- **测试集** = 真正的考试，只能用一次，不能拿来练手；
- **过拟合** = 题目背得滚瓜烂熟，但换一道新题就抓瞎——模型在训练集上表现很好，在测试集上表现很差；
- **Dropout 和正则化** = 故意在复习时遮住部分笔记，逼自己真正理解，而不是死记硬背。

参考答案与详细解析

(1) 独热编码的原理及必要性：

独热编码将 k 类的类别变量转换为 k 维二进制向量，每次只有一个维度为 1，其余为 0。例如”北京 =1，上海 =2，广州 =3”→”北京 =[1,0,0]，上海 =[0,1,0]，广州 =[0,0,1]”。

不能直接用整数编码的原因：神经网络会把整数值当作连续数值处理，隐含”北京离上海的距离 =1，北京离广州的距离 =2”这样的大小关系，而三个城市在特征语义上本无大小之分，这种人为引入的大小关系会误导模型学习错误的规律。

(2) 标准化的必要性：

梯度下降对各特征的尺度非常敏感。面积范围 $[50, 500]$ 和房间数范围 $[1, 10]$ 相差 50 倍，对应的梯度幅度差异巨大：面积特征上的梯度可能比房间数的梯度大 50 倍。这导致优化时只能使用极小的学习率（否则面积方向震荡），房间数方向更新极慢，整体收敛速度大幅下降，甚至难以收敛。标准化后两者都在均值 0、方差 1 的尺度上，梯度幅度一致，可以使用更大的学习率，收敛更稳定。

(3) 输出层无激活函数的原因 & 分类改造：

房价预测是回归任务，预测值应该是任意正实数（如 200 万、580 万），若加 Sigmoid 输出只能在 $(0, 1)$，若加 ReLU 又会引入非必要的截断。因此输出层直接保留线性输出，不加激活函数。

改为二分类任务的修改：

```python
# 修改方案一（推荐）：输出 logit，配合 BCEWithLogitsLoss
nn.Linear(128, 1)   # 输出层不加激活函数
loss_fn = nn.BCEWithLogitsLoss()  # 内置 Sigmoid

# 修改方案二：输出概率，配合 BCELoss
```

```
nn.Sequential(nn.Linear(128, 1), nn.Sigmoid())
loss_fn = nn.BCELoss()

```

(4) 三个步骤的作用：

- `optimizer.zero_grad()`：清空所有参数上一次迭代积累的梯度。PyTorch 默认将梯度**累加**而不是覆盖，若不清零，本次梯度会叠加在历史梯度上，导致参数更新方向错误；
- `loss.backward()`：触发自动微分，从损失出发反向计算所有参数的梯度，存储在各参数的 `.grad` 属性中；
- `optimizer.step()`：读取各参数的 `.grad`，按优化算法（如 Adam）更新参数值。

`zero_grad()` 必须在 `backward()` 之前，否则新的梯度会累加在旧梯度上，使得参数更新错误，训练无法正常进行。

(5) RMSE 评估指标：

实际中常用均方根误差（RMSE，Root Mean Squared Error）：

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2} = \sqrt{\text{MSE}}$$

优势：单位与房价相同（元而非元的平方），直觉上易于理解——"模型预测误差平均约为 X 元"。此外，有时还使用平均绝对百分比误差（MAPE）衡量相对误差：

$$\text{MAPE} = \frac{1}{N}\sum_{i=1}^{N}\left|\frac{\hat{y}_i - y_i}{y_i}\right| \times 100\%$$

房价预测完整训练循环模板

```python
for epoch in range(num_epochs):
    model.train()                    # 切换训练模式
    for X_batch, y_batch in dataloader:
        optimizer.zero_grad()        # 第一步：清空梯度
        pred = model(X_batch)        # 第二步：前向传播
        loss = loss_fn(pred, y_batch)# 第三步：计算损失
        loss.backward()              # 第四步：反向传播
        optimizer.step()             # 第五步：更新参数

    model.eval()                     # 切换推理模式
    with torch.no_grad():            # 不计算梯度
        val_pred = model(X_val)
        val_loss = loss_fn(val_pred, y_val)
    print(f"Epoch {epoch}: val_loss={val_loss:.4f}")

```

现实应用场景

神经网络房价预测——实际中比想象中更复杂：
Zillow（美国最大房产平台）的 Zestimate 模型，就是用神经网络估算房价的商业落地。它面对的挑战包括：特征高达数百个（面积、朝向、学区评分、周边设施……），且各地区特征分布差异极大（北京的均价和县城均价不能用同一套权重）。实际中会用到更复杂的技巧，如地理位置嵌入（把经纬度变成向量）、特征交叉、对目标值取对数（log(SalePrice)）来压缩量纲。但本章案例的全流程骨架——特征工程 → 建模 → 选损失函数 → 训练 → 评估——与工业级方案完全一致。

2 本章小结：把零件装成一台可以跑的机器

学完这 6 道题，如果还觉得”概念飘在空中”，试试把整章内容浓缩成这几句画面：

- **激活函数**：ReLU 是隐藏层的标配；Sigmoid 输出概率（二分类）；Softmax 输出概率分布（多分类）；回归输出层什么都不加。
- **参数初始化**：绝对不能全初始化为同一个值；ReLU 配 Kaiming，Sigmoid/Tanh 配 Xavier。
- **损失函数**：分类用交叉熵，回归用 MSE/RMSE；`CrossEntropyLoss` 自带 Softmax，不要重复。
- **优化器**：大多数任务直接用 Adam；需要精调时，学习率衰减是让训练最终收敛的关键。
- **模型结构**：简单的用 `Sequential`，需要灵活结构的用 `nn.Module` 子类；`forward` 里追踪每层输出的形状是调试的第一步。
- **训练循环**：`zero_grad → backward → step`，顺序不能乱；训练时 `model.train()`，推理时 `model.eval()`，别忘了切换。

给零基础读者的一句总比喻

如果把训练一个神经网络比作培养一名厨师：

- **激活函数** = 厨师的直觉（非线性判断力）：没有它，再多的训练也只会做直来直去的”线性菜”；
- **初始化** = 开业前的准备状态：从一个好的起点出发，比从乱糟糟的状态开始训练省很多时间；
- **损失函数** = 顾客的反馈表单：反馈维度选对了，厨师才知道往哪个方向改进；
- **优化器** = 厨师改进菜谱的方法论：是把每次差评都死记硬背（SGD），还是总结规律系统改进（Adam）？
- **最终效果**：一个调好参数、训练充分的神经网络，就像一位经过千百次反馈迭代、炉火纯青的大厨——它”记住”的不是具体的食谱，而是做好菜的底层规律。